LangGraph Workflows Demo

Dr. Sunil Saumya

Dept of Data Science & Artificial Intelligence
Indian Institute of Information Technology Dharwad

sunilsaumya @iiitdwd.ac.in



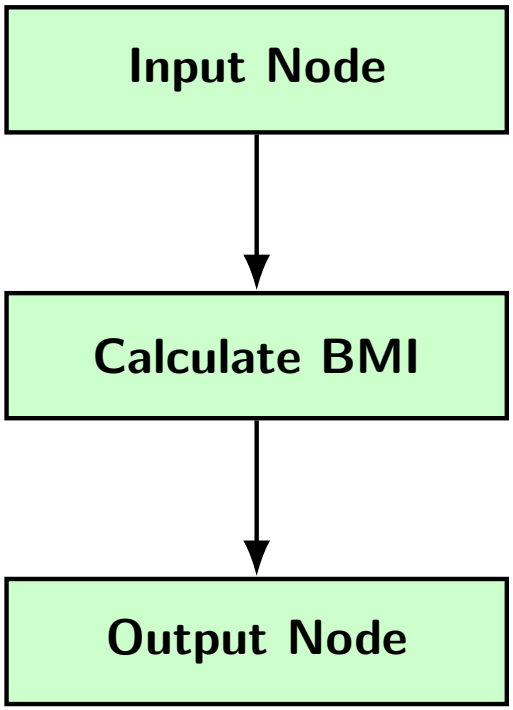
September 4, 2026

Outline

1. Sequential Workflow
   - Non-LLM Workflow
   - LLM Workflow
   - Prompt Chaining Sequential Workflow
2. Parallel Workflow
   - Non-LLM Workflow
   - LLM Workflow
3. Conditional Workflow
   - Non-LLM Workflow
   - LLM Workflow
4. iterative Workflow
   - LLM Workflow

Sequential Workflow

Example: BMI Calculator Workflow in LangGraph

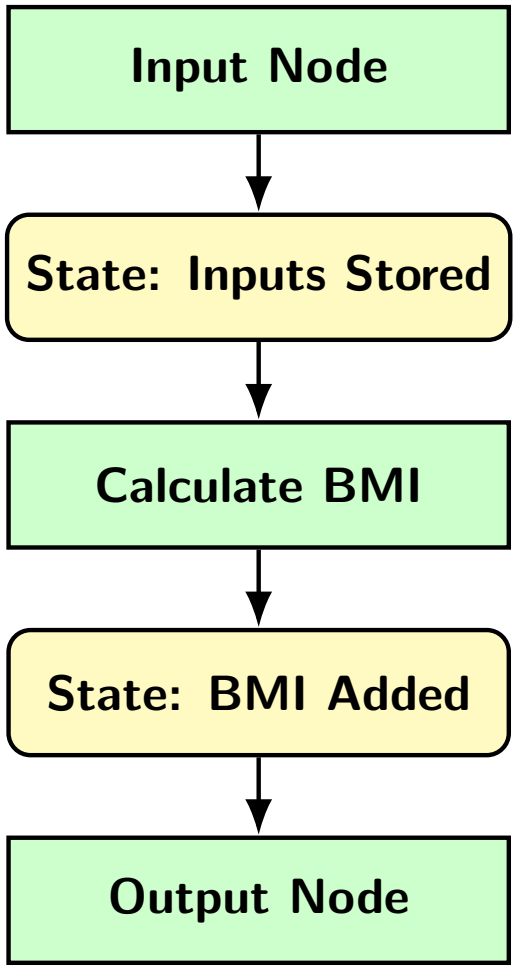


State Dictionary

Initial State:

```
{
  "weight": float,
  "height": float,
  "bmi": float
}
```

Example: BMI Calculator Workflow in LangGraph



After Input Node:

```
{
  "weight": 70,
  "height": 1.75,
  "bmi": None
}
```

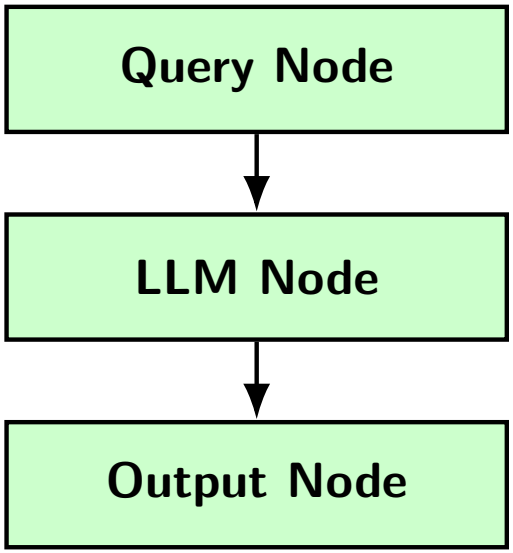
After Calculate BMI:

```
{
  "weight": 70,
  "height": 1.75,
  "bmi": 22.86
}
```

Demo:- BMI Calculator Workflow

Each node updates or appends values in the state dictionary - enabling data flow and persistence across the workflow.

Example: LLM QA Workflow in LangGraph

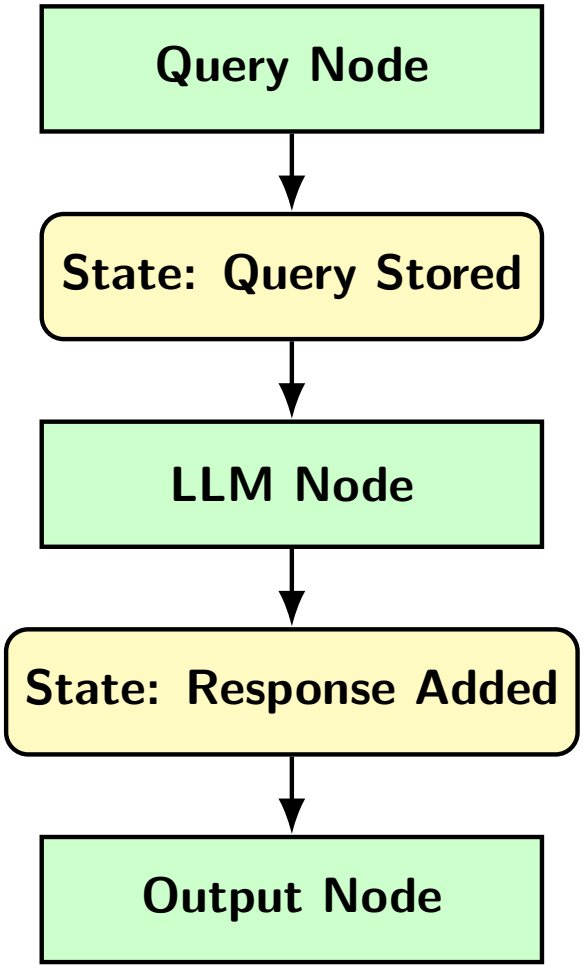


State Dictionary (Evolution)

Initial State:

```
{
  "query": str,
  "response": str
}
```

Example: LLM QA Workflow in LangGraph



After Query Node:

```
{
  "query": "What is LangGraph?",
  "response": None
}
```

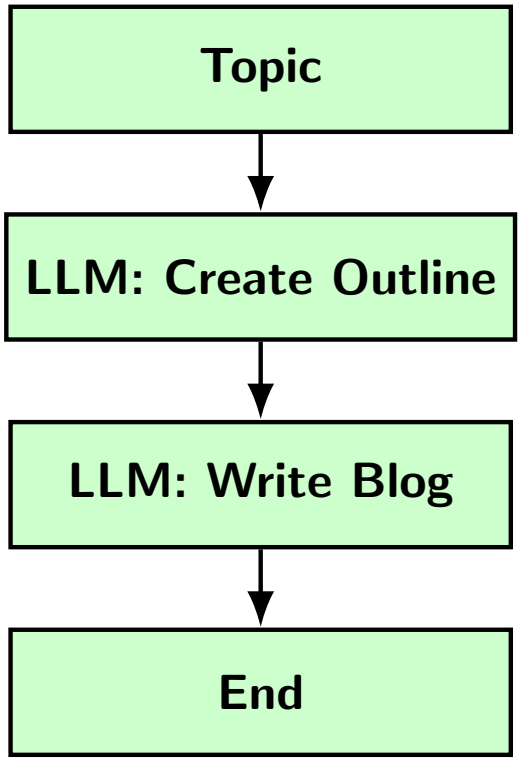
After LLM Node:

```
{
  "query": "What is LangGraph?",
  "response": "LangGraph is a framework
  for building stateful LLM workflows."
}
```

Demo:- LLM QA Workflow

Each node reads from and updates the shared state - maintaining continuity across the workflow.

Example: Prompt Chaining Workflow in LangGraph

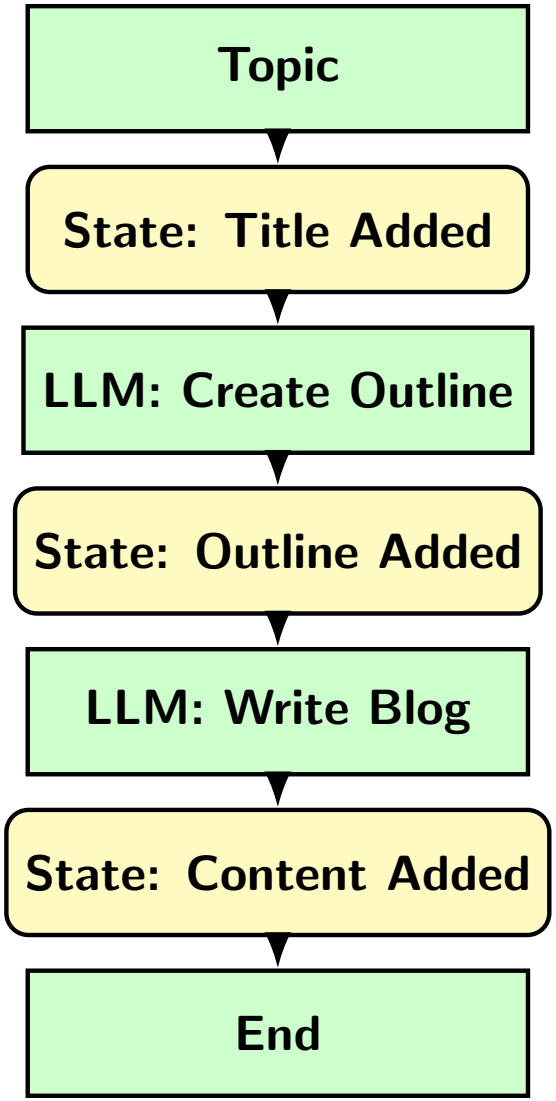


Initial State:

```
{
  "topic": None,
  "outline": None,
  "content": None
}
```

Demo:- Prompt chaining workflow

Example: Prompt Chaining Workflow in LangGraph



After Topic Node:

```
{
  "topic": "AI for Healthcare",
  "outline": None,
  "content": None
}
```

After Outline Node:

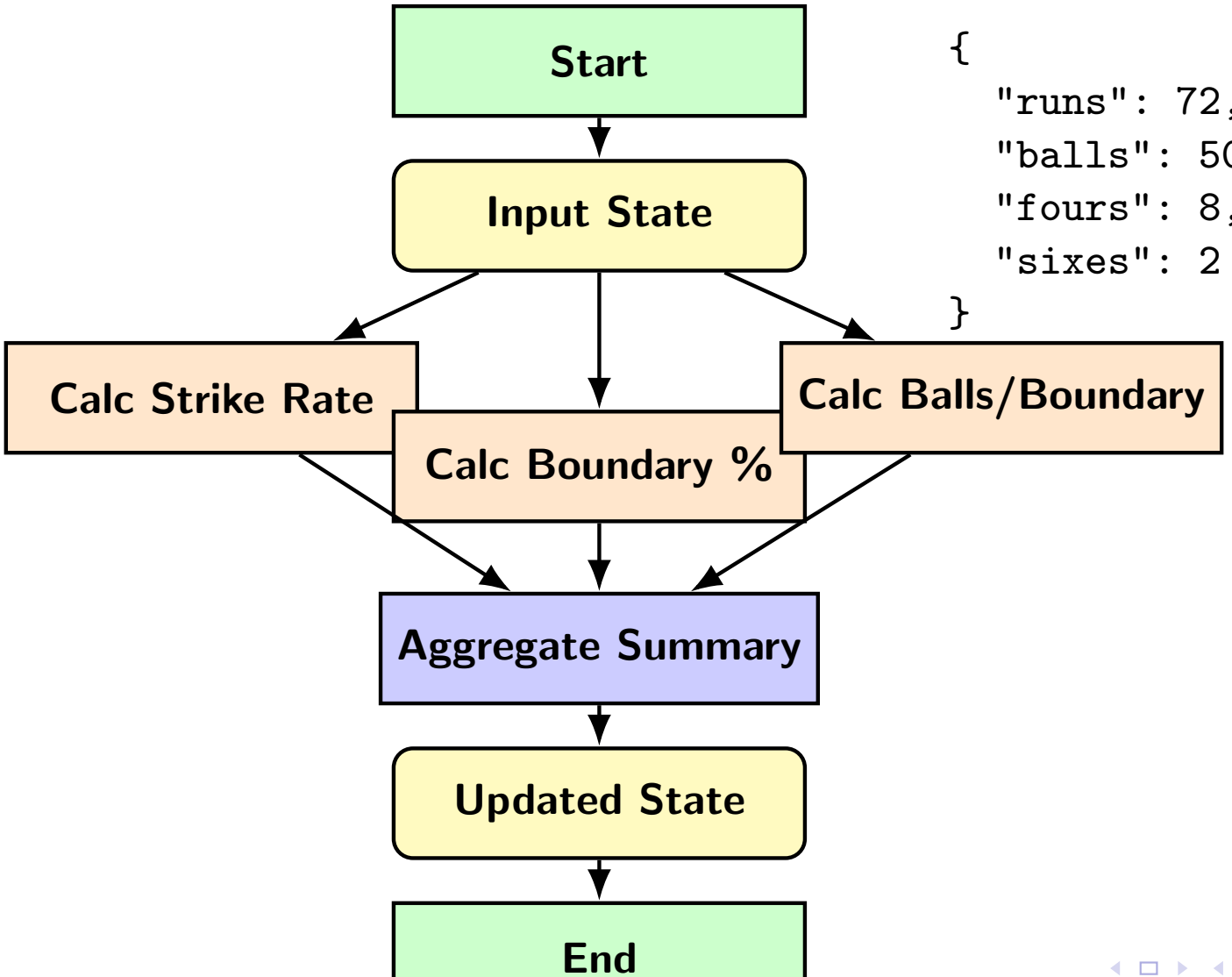
```
{
  "topic": "AI for Healthcare",
  "outline": "1. Intro
2. Use Cases
3. Challenges",
  "content": None
}
```

After Blog Node:

```
{
  "topic": "AI for Healthcare",
  "outline": "...",
  "content": "Full blog generated by LLM."
}
```

Parallel Workflow

Parallel Workflow Example: Cricket Inning Analysis



Initial State:

```
{
  "runs": 72,
  "balls": 50,
  "fours": 8,
  "sixes": 2
}
```

Parallel Workflow Example: Cricket Inning Analysis

State Dictionary (Evolution)

After Parallel Computations:

```
{
  "runs": 72,
  "balls": 50,
  "fours": 8,
  "sixes": 2,
  "strike_rate": 144.0,
  "boundary_percent": 61.1,
  "balls_per_boundary": 5.0
}
```

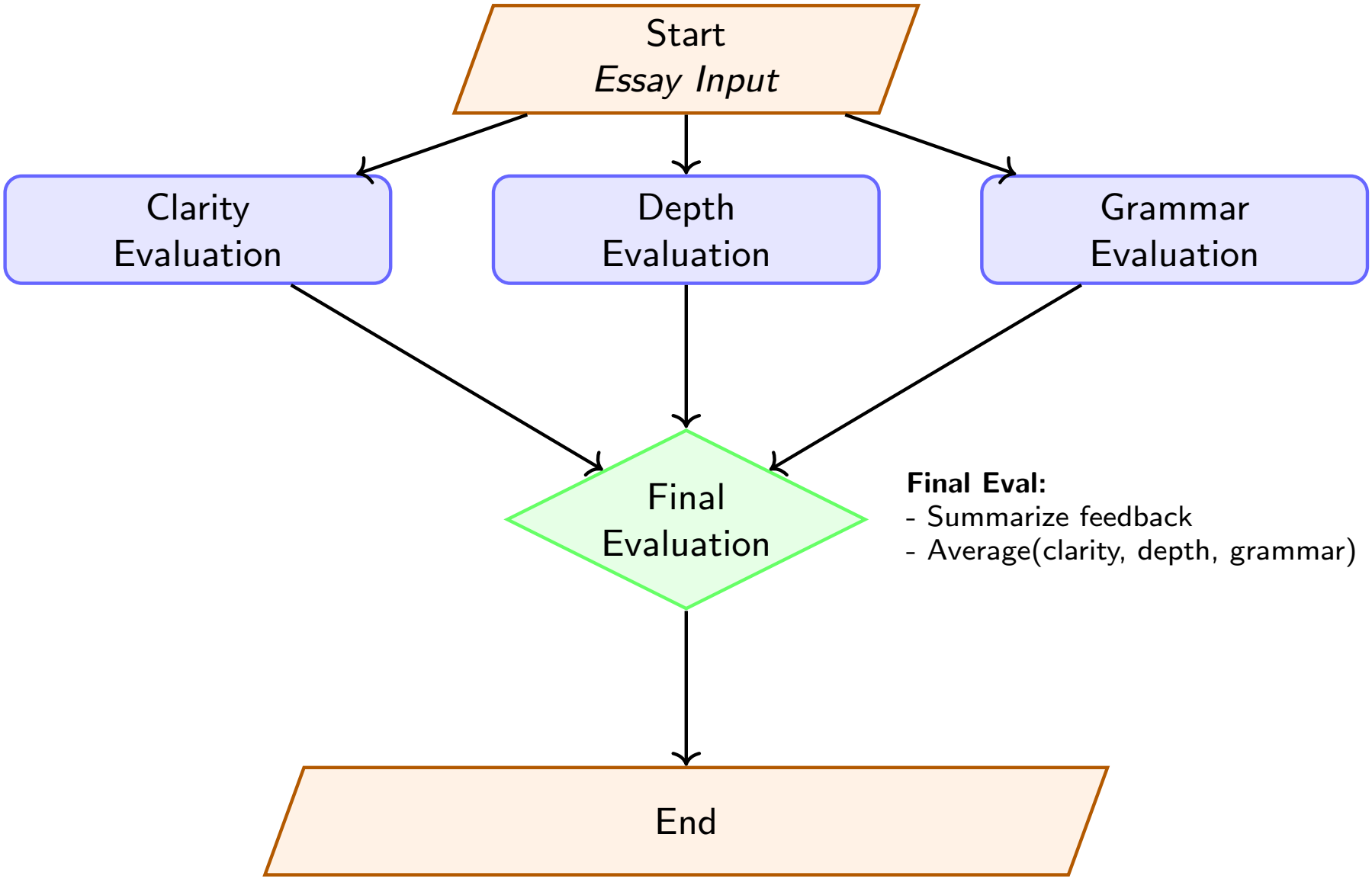
After Summary Node:

```
{
  "summary":
    "Scored 72 (50) with SR 144.0,
     61.1% runs in boundaries."
}
```

Each metric node runs in parallel, updating the shared state before aggregation.

Demo:- Non-LLM Parallel Workflow

Parallel Workflow: Essay Evaluation



Parallel Workflow: Essay Evaluation

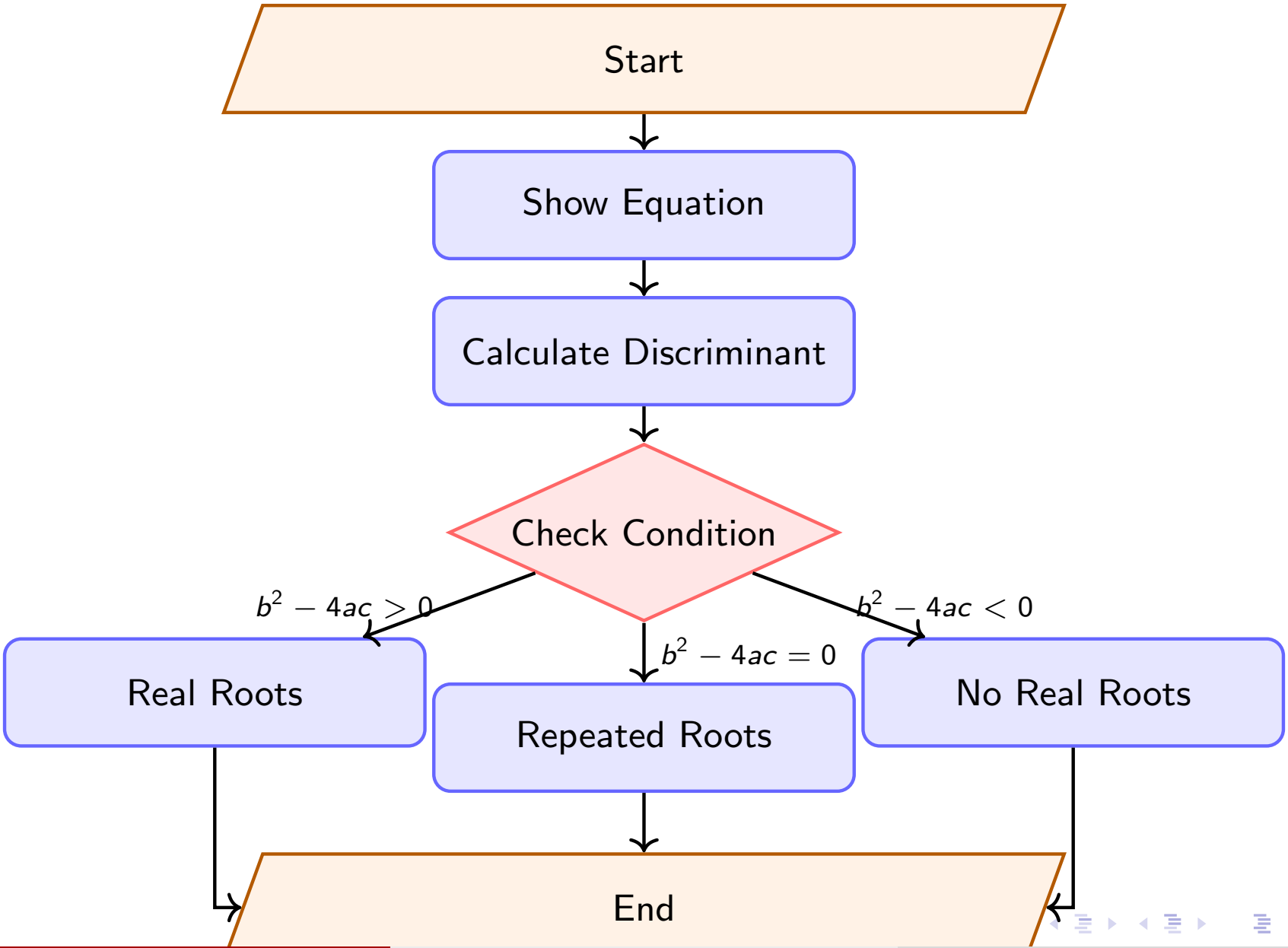
States (Dictionary)

```
{
  "essay": str,
  "clarity_feedback": str,
  "clarity_score": int,
  "depth_feedback": str,
  "depth_score": int,
  "grammar_feedback": str,
  "grammar_score": int,
  "summary_feedback": str,
  "final_score": float
}
```

Demo:- Non-LLM Parallel Workflow

Conditional Workflow

Conditional Workflow: Quadratic Equation Solver



Conditional Workflow: Quadratic Equation Solver

States (Dictionary)

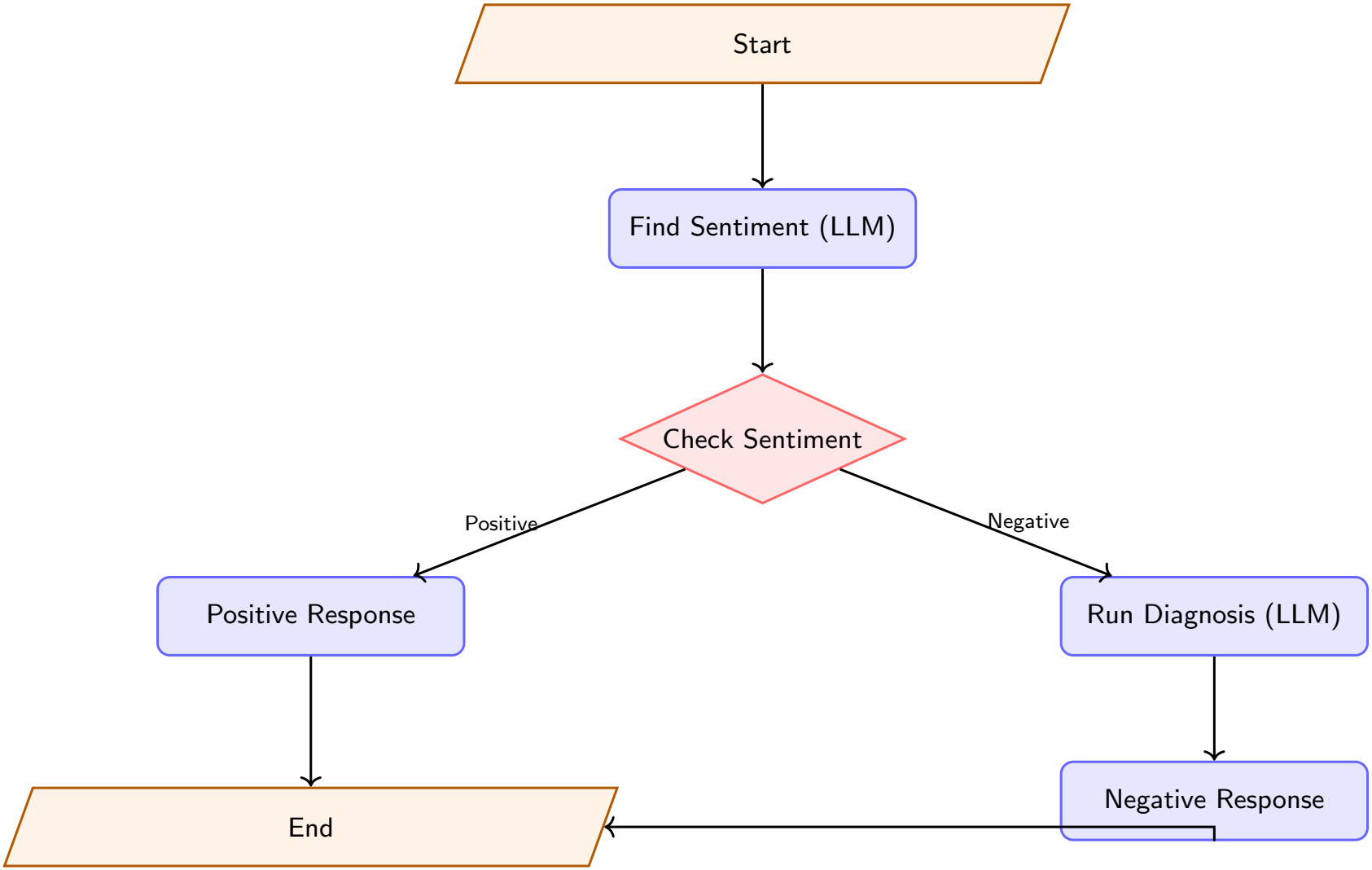
```
{
  "a": float,
  "b": float,
  "c": float,
  "equation": str
  "discriminant": float,
  "result": str
}
```

Condition Logic:

```
if D > 0: real_roots
elif D == 0: repeated_roots
else: no_real_roots
```

Demo:- Non-LLM Conditional Workflow

LLM-Based Conditional Workflow: Sentiment Review Flow



Demo:- LLM Conditional Workflow

ReviewState: Data Flow and Evolution

```
class ReviewState(TypedDict):
    review: str
    sentiment: Literal["positive", "negative"]
    diagnosis: dict[str, Any]   # issuetype, tone, urgency
    response: str
```

Example Initialization

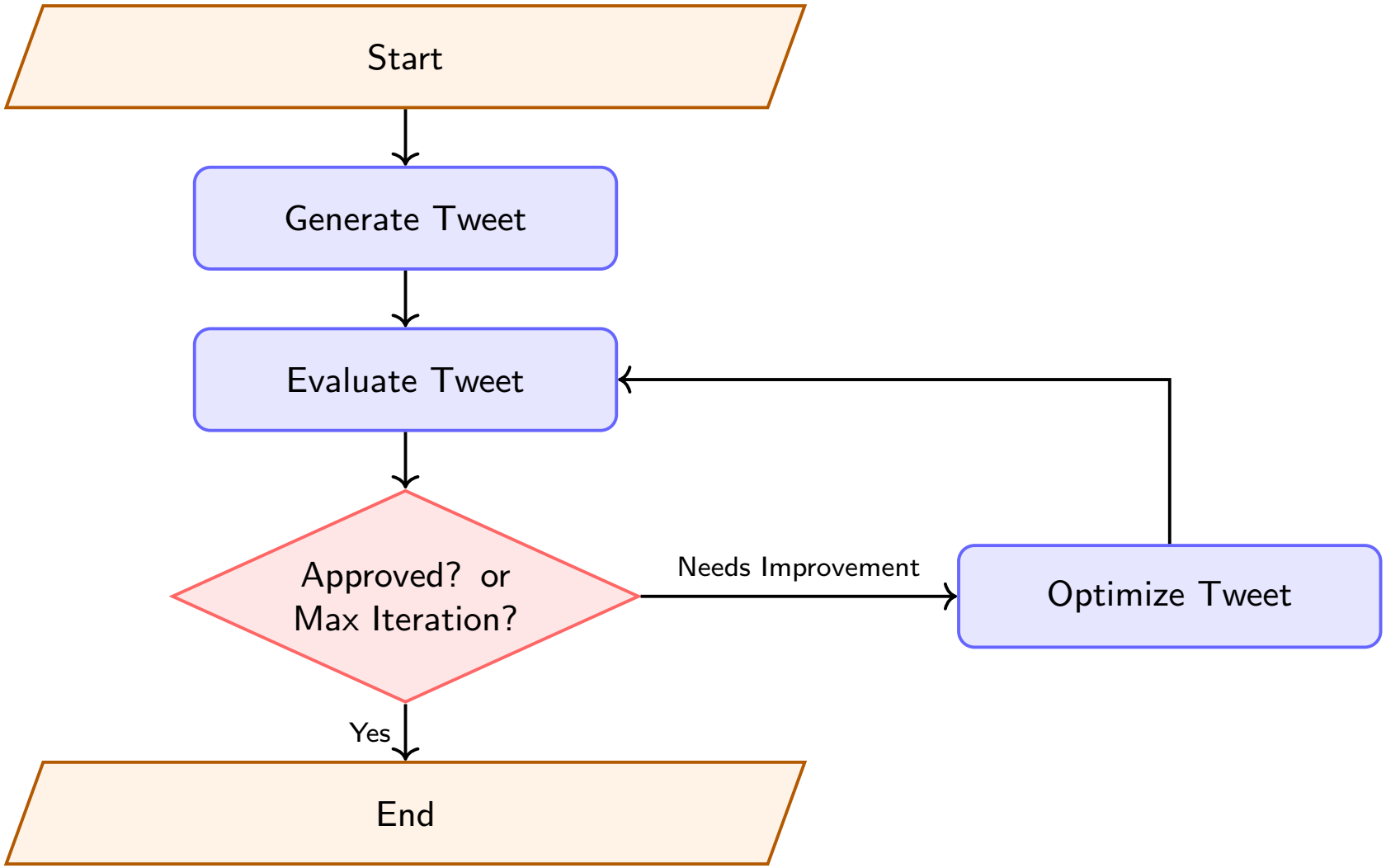
```
state = {
  "review": "The product was slow and buggy.",
  "sentiment": None,
  "diagnosis": {},
  "response": ""
}
```

State Evolution (Sample Run)

Node	State Update
find_sentiment	sentiment = "negative"
run_diagnosis	diagnosis = { "issuetype": "performance", "tone": "frustrated", "urgency": "high" }
negative_response	response = "We're sorry... Our team will fix it soon."
Final State	{"sentiment":"negative", "response": "..."}

Iterative Workflow

Iterative Workflow: Tweet Refinement Loop



TweetState: Data Flow and Iterative Refinement

```
class TweetState(TypedDict):
    topic: str
    tweet: str
    evaluation: Literal["approved", "needs_improvement"]
    feedback: str
    iteration: int
    max_iteration: int
    tweet_history: Annotated[list[str], operator.add]
    feedback_history: Annotated[list[str], operator.add]
```

State Evolution (Sample Run)

Node	State Update
generate	tweet = "AI saves lives..."
evaluate	evaluation = "needs_improvement"
optimize	feedback = "Too generic" iteration += 1; tweet_history += [tweet] feedback_history += [feedback]
evaluate (2nd pass)	evaluation = "approved"
Final State	iteration = 2, evaluation = "approved"

Demo:- LLM Iterative Workflow

Thank You!